27. a. Singly Linked List – TraverseListReverseOrderUsingStack –Program Analysis and Time Complexity .

***Program*: –**

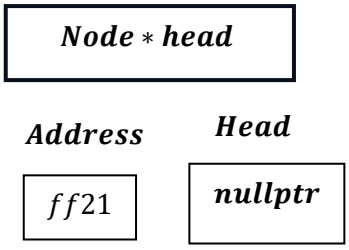
```
#include <stack>
//#include <bits/stdc++.h>
…..
void TraverseListReverseOrderUsingStack(Node **head)
{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    stack<Node *> s;
    Node **current = head;
    while (*current != nullptr)
    {
        s.push(*current);
        current = &((*current)->next);
    }
    while (!s.empty())
    {
        cout << s.top()->data << " ";
        s.pop();
    }
    cout << endl;
}
…case 25: {
            cout << endl;
            cout << endl;
            cout << "Traverse List Reverse Order Iterative Using Stack" << endl;
            TraverseListReverseOrderUsingStack(&head);
            cout << endl;
            cout << endl;
            break;
        }
```

$\#include < stack >$ *is a container adaptor with LIFO(Last In First Out)type of working, mechanism and it is a library inside Standard Template Library(STL).*

Such mechanism also falls under $\#include < bits/stdc++.h >$ *, and this header file is not part of the C++ standard.*

1) *Linked List is Empty*:

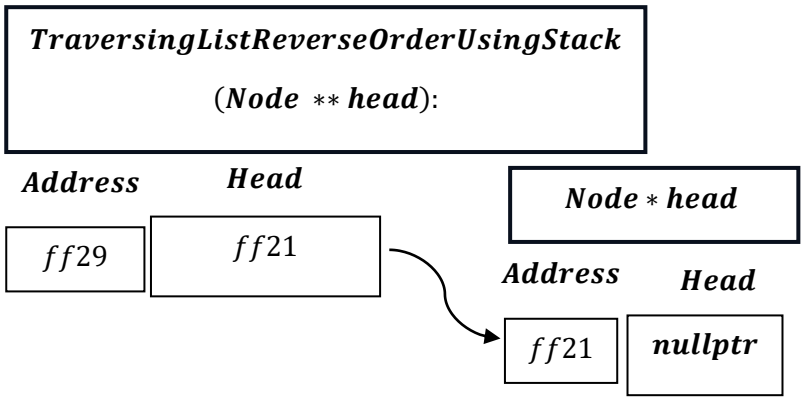
Pictorially depicted as:




```
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
```

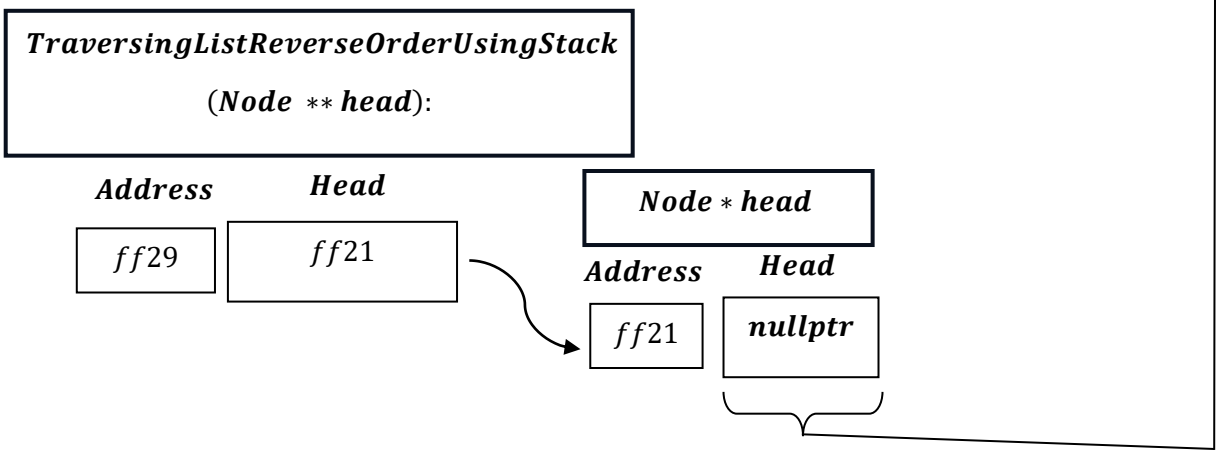
TraversingListReverseOrderUsingStack(&***head***: ***ff21***);

```
void TraverseListReverseOrderUsingStack (Node **head) {….}
```



$if(*head == nullptr)$ *if it is true*

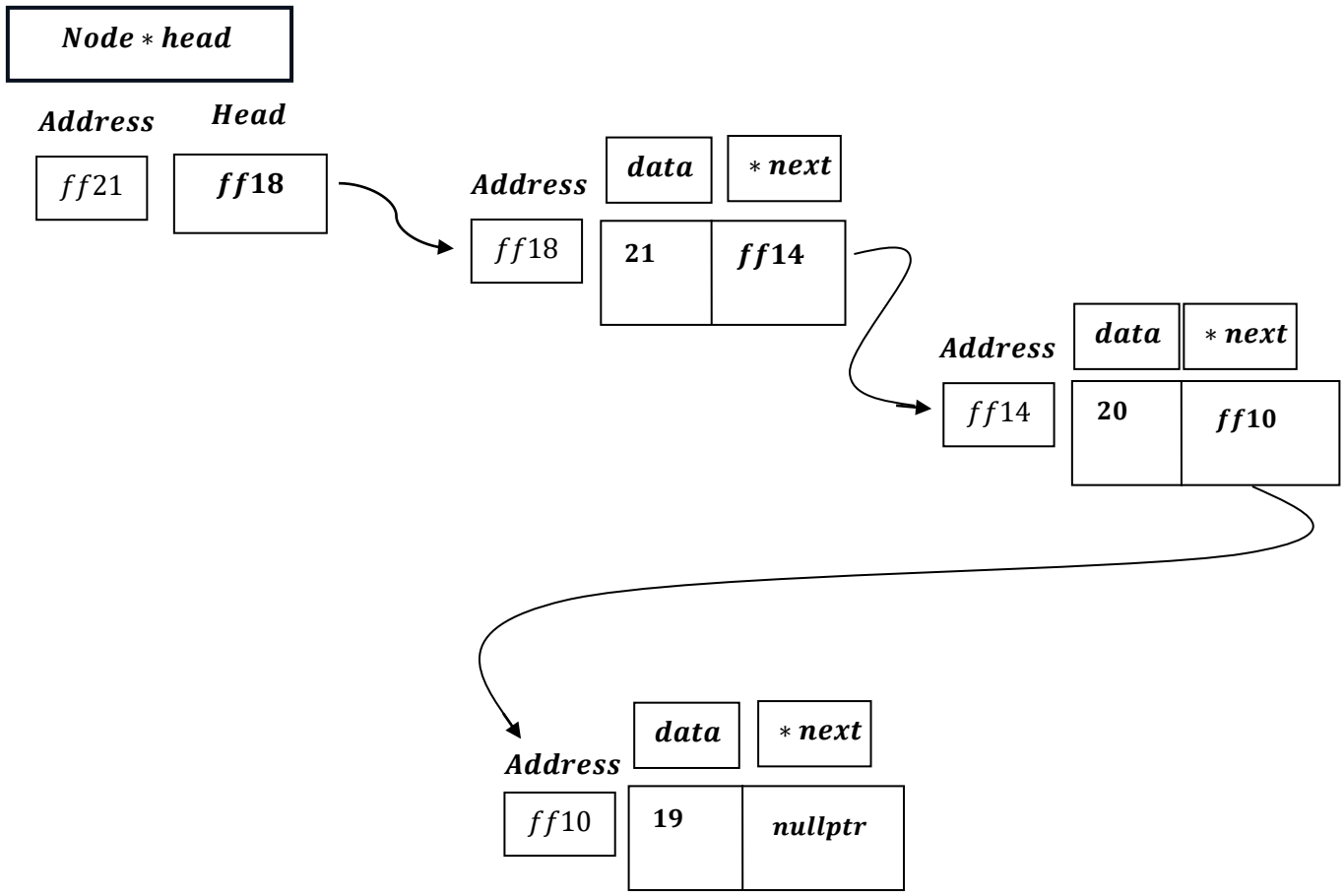
If true then Why true ?
Ans: Though $*$ head which is of Node $**$ head
temporary pointer to pointer variable of the function
, as $*$ head is used , it dereference from Node $**$ head and
refer to Node $*$ head global pointer to which Node $**$ head
is pointing to and then its value is checked is
whether equal to nullptr or not ? This occurs during execution
and compilation of the `TraversingListReverseOrderUsingStack` function.



If List is Empty , then , it prints a message : `Error: List is empty.` and returns nothing as function has void data type . And function ends.

---

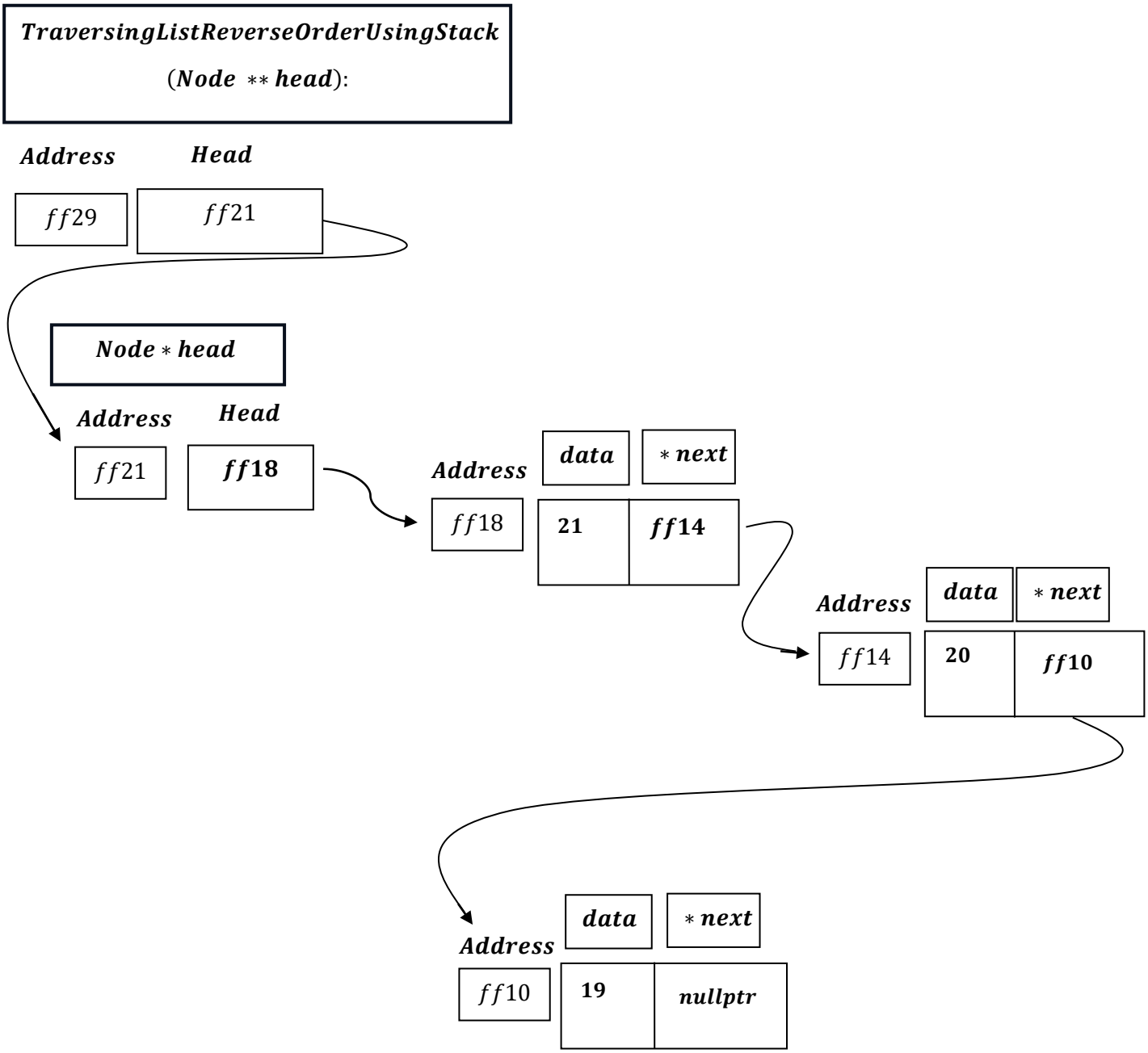
2)*Linked List is not Empty*:



Function Call:

```
TraverseListReverseOrderUsingStack(&head);
```

$i.e.\mathbf{TraverseListReverseOrderUsingStack}(\&\mathbf{head}: \mathbf{ff21})$

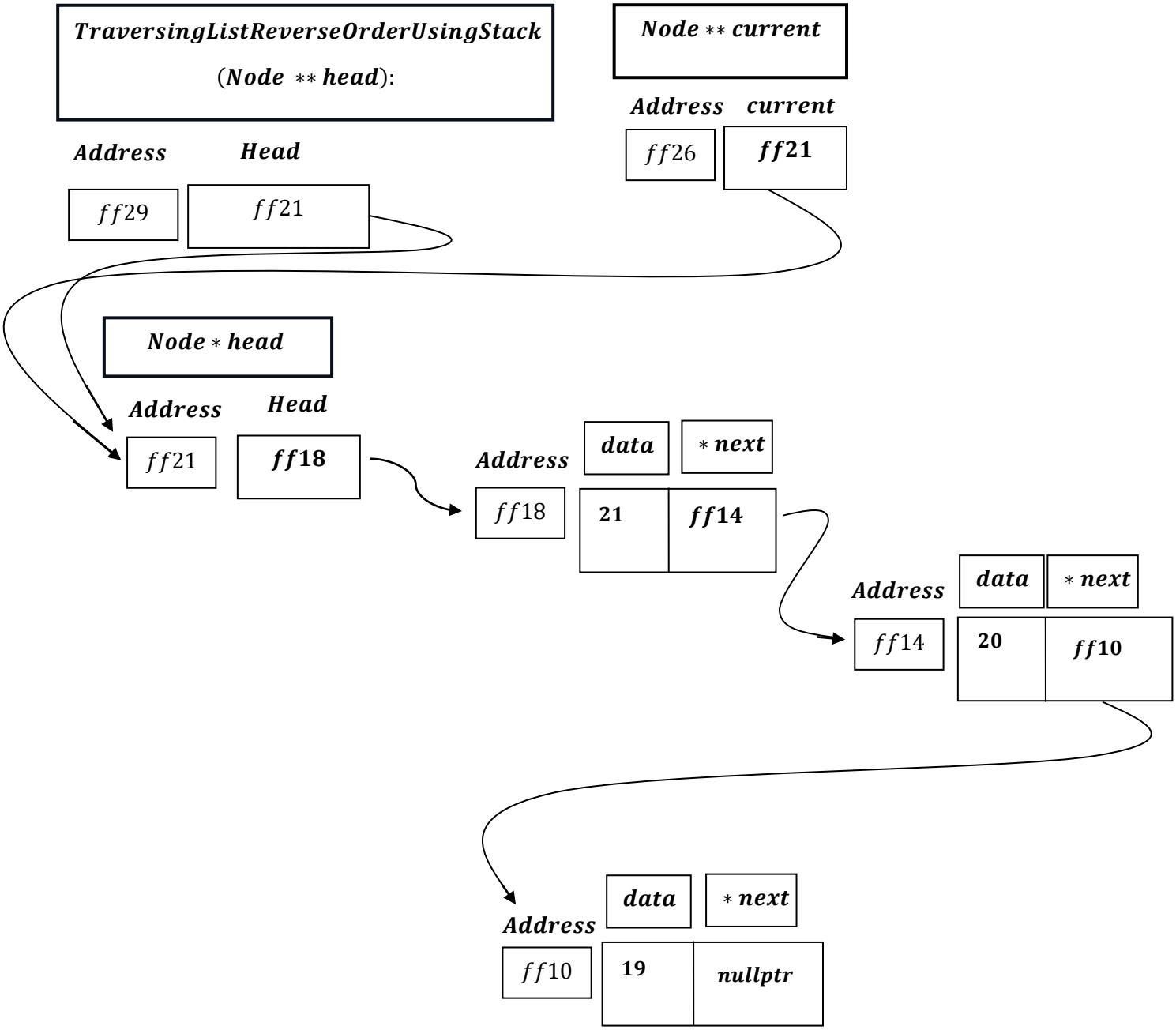


Now,

```
stack<Node *> s;
```

*stack < Node *> s; declares a stack named 's' that is designed to hold pointers to Node objects. Next:*

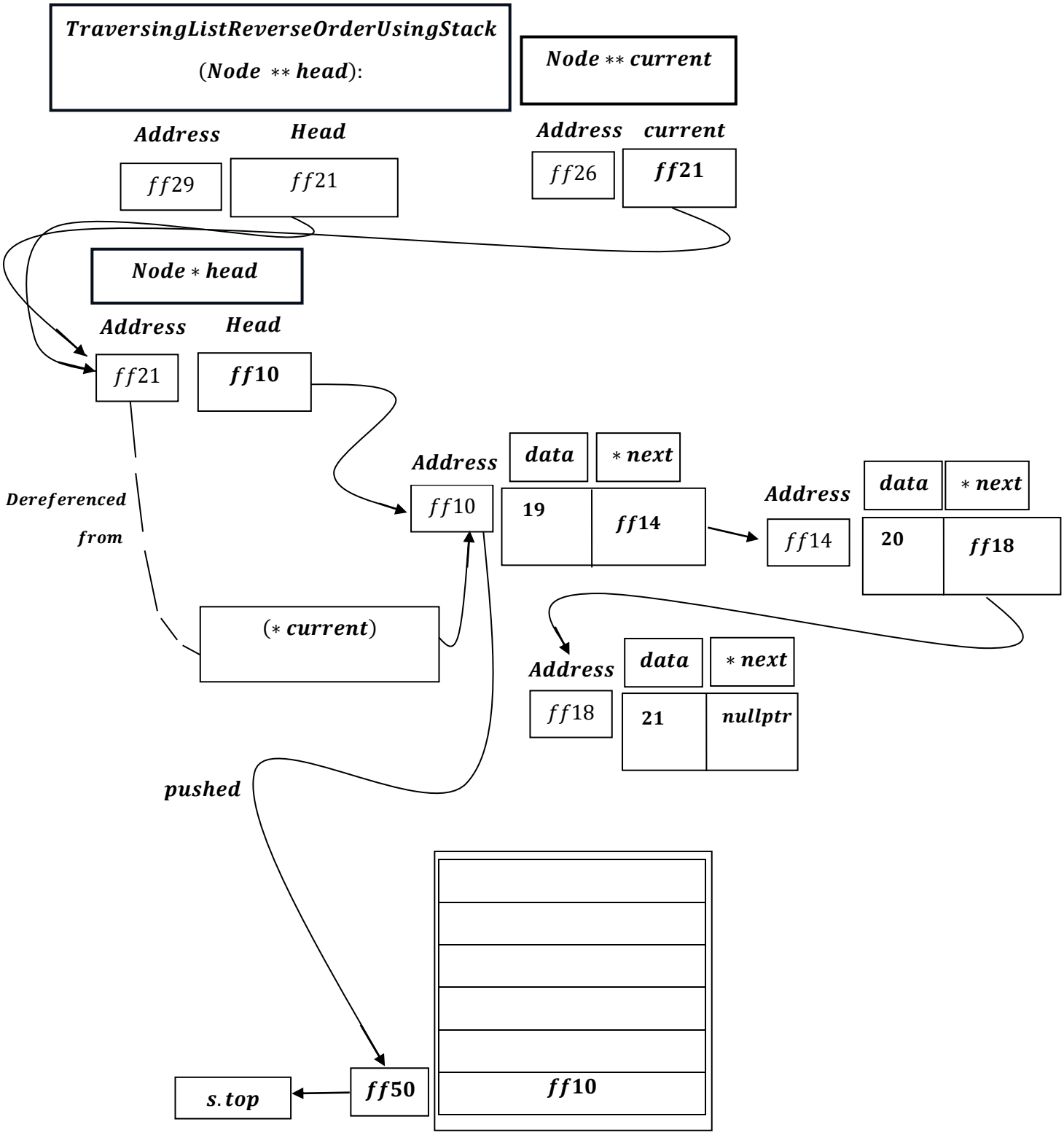
```
Node **current = head;
```




```
    while (*current != nullptr)
    {
        s.push(*current);
        current = &((*current)->next);
    }
```

$*$ ***current is not equal to*** $(!=)$ ***nullptr which is true then***:

```
        s.push(*current);
```



$s.push(*current)$, here $*current$ is Node $*$ $\underline{*current}$ pointer , and dereferencing pointer $(*)$dereference it , to get the Node $*$ it points to. Here $\underline{Node *}$ points to address: $ff10$, where

$\underline{current}$ points to address : $ff21$. Hence as $\underline{*current}$ pointer gets deferenced by $*$ (dereferencing operator), i.e. now

no longer pointing to address : $ff21$, hence , now, Node $*$ pointed

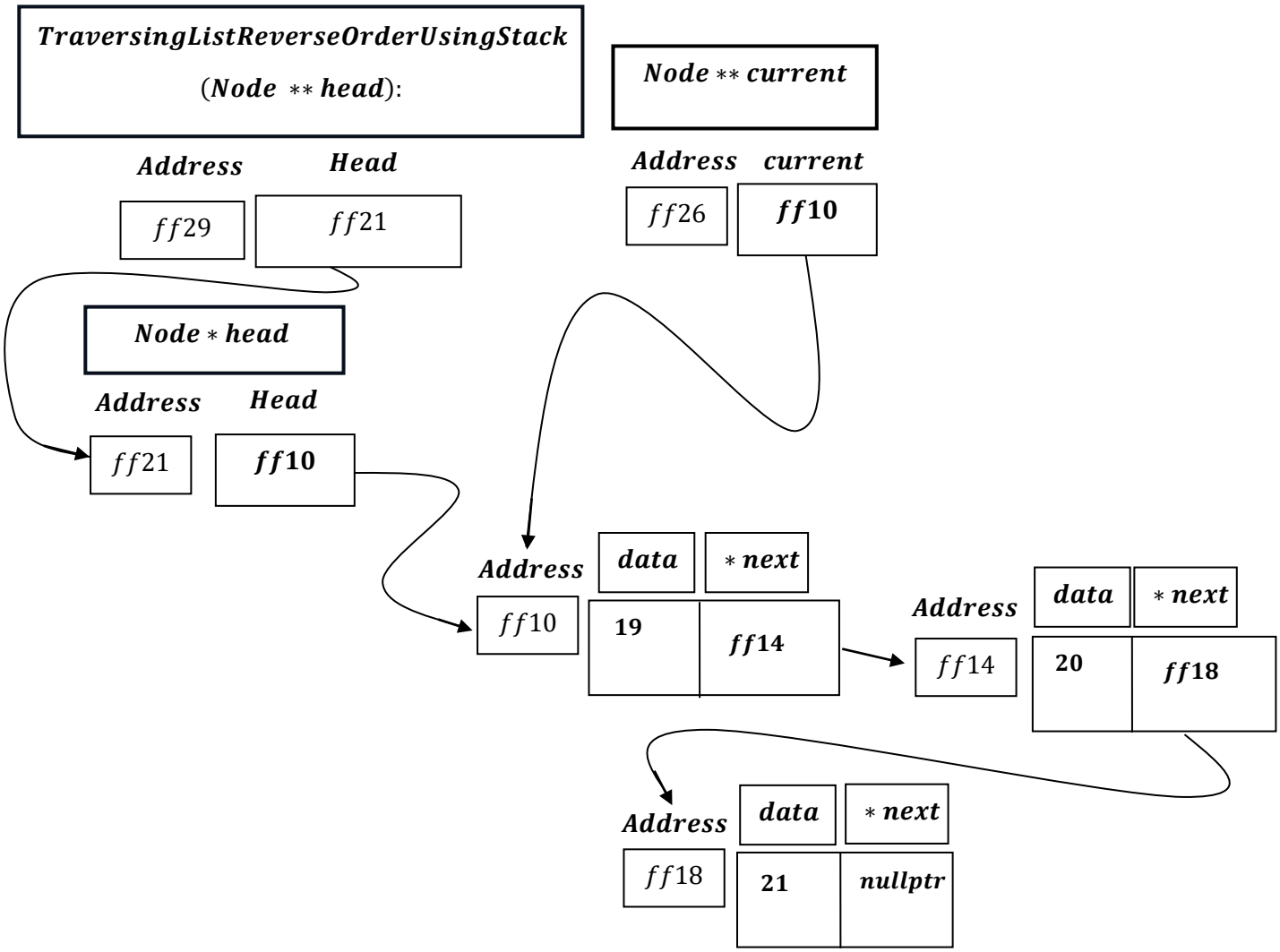
address is referenced i.e. $ff10$.

$*$ currrent is $ff10$, which is address of the first node: [Address: $ff10$][data: 19| $*$ next: $ff14$]

The stack will store this address ($ff10$), which references the first node.

Now updation,

```
        current = &((*current)->next);
```



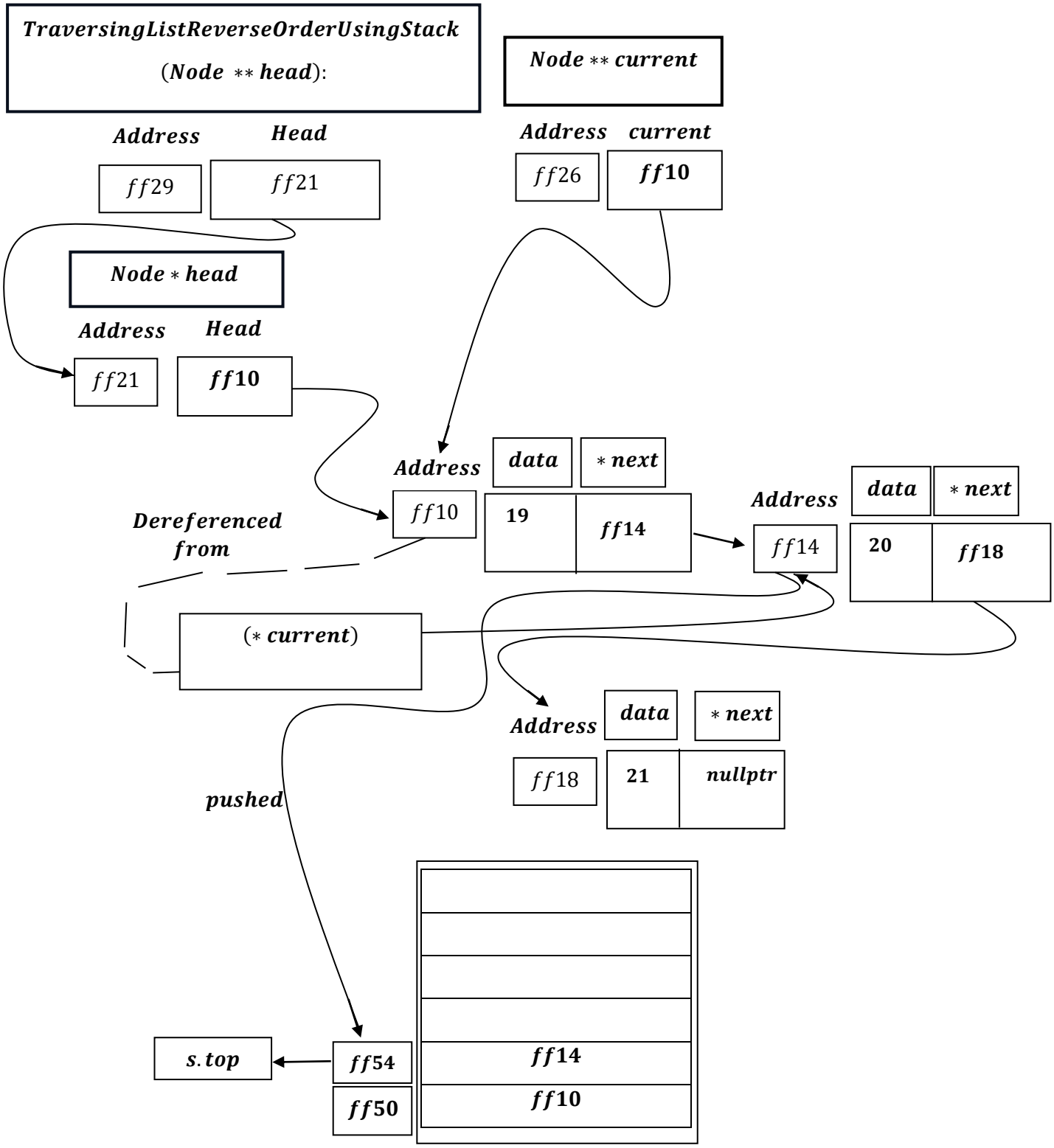
During the first iteration, ($*$ current) dereferences current to get the Node $*$ it points to, which is the first node in the list. The expression ($*$ current) $\rightarrow$ next

$, where\ current\ points\ to\ address\ of\ Node * Head : ff21, but\ (* current)$ $dereferences\ first\ and\ now\ Node * points\ to\ first\ node, hence\ (* current) \rightarrow next,$ $will\ point\ to\ next\ pointer\ of\ first\ node\ i.e. ff14\ \ .The\ \&\ operator\ then\ takes\ the$ $address\ where\ this\ next\ pointer\ is\ stored\ i.e. the\ address: ff10, where\ \ ff14\ is$ $stored\ . Hence, current = \&\big((* current) \rightarrow next\big) = ff10\ under\ which\ ff14\ is\ stored.$

```
    while (*current != nullptr)
    {
        s.push(*current);
        current = &((*current)->next);
    }
```

$* current\ is\ not\ equal\ to\ (! =)\ nullptr\ which\ is\ true\ then:$

```
        s.push(*current);
```



$s.push(*current)$, here $*current$ is Node $*$ $\underline{*current}$ pointer

, and dereferencing pointer $(*)$dereference it , to get the

Node $*$ it points to. Here $\underline{Node *}$ points to address: $ff14$, where

$\underline{current}$ points to address : $ff10$. Hence as $\underline{*current}$ pointer gets deferenced by $*$ (dereferencing operator), i.e. now

no longer pointing to address : $ff10$, hence , now, Node $*$ pointed

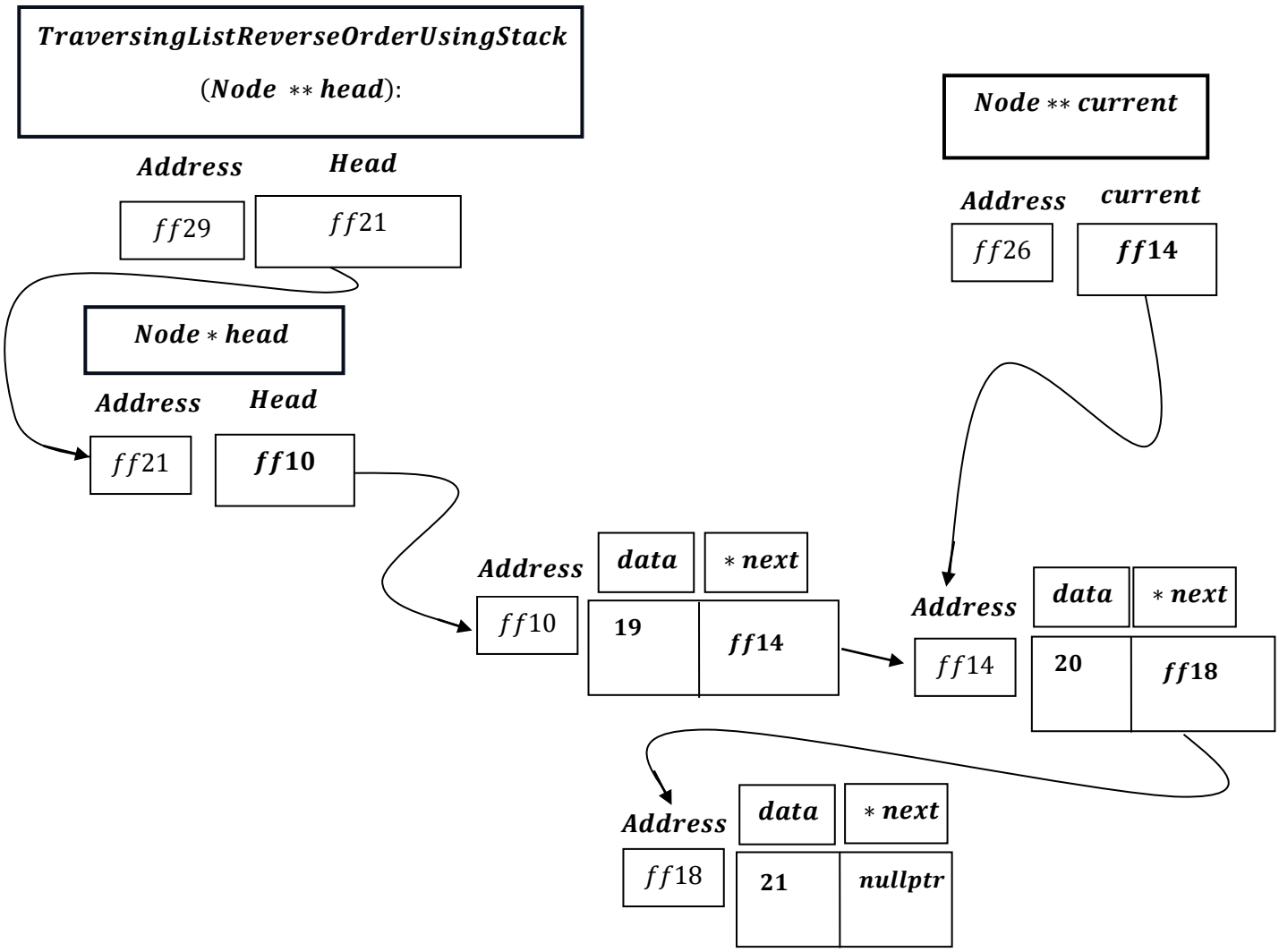
address is referenced i.e. $ff14$.

$*$ currrent is $ff14$, which is address of the 2nd node: [Address: $ff14$][data: 20| $*$ next: $ff18$]

The stack will store this address ($ff14$), which references the second node.

Now updation,

```
        current = &((*current)->next);
```



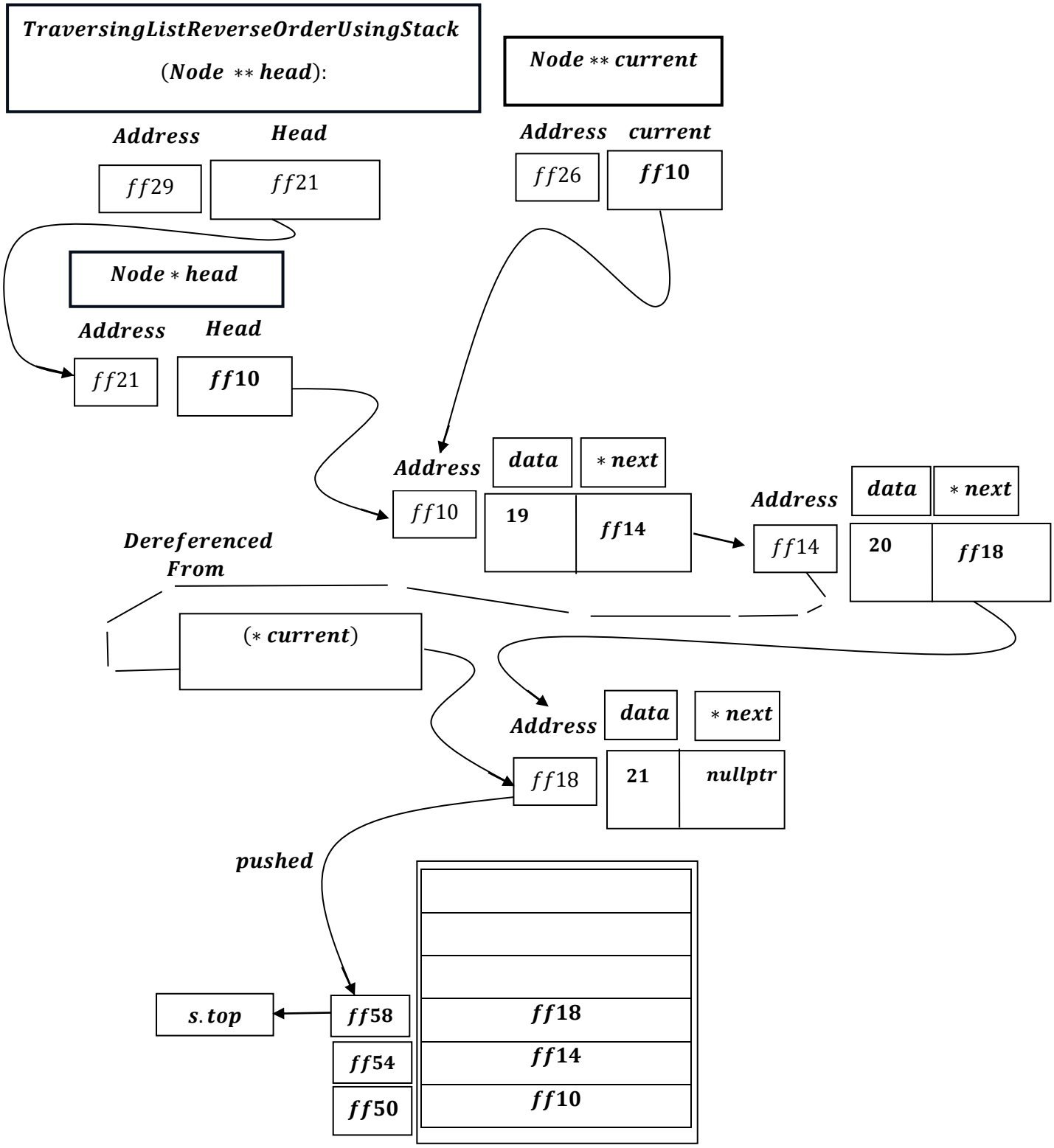
During the second iteration, ($*$ current) dereferences current to get the Node $*$ it points to, which is the second node in the list. The expression ($*$ current) $\rightarrow$ next

$, where\ current\ points\ to\ address: ff10, but\ (*current)\ dereferences\ first\ and$
$now\ Node*\ points\ to\ second\ node, hence\ (*current) \rightarrow next, will\ point\ to$
$next\ pointer\ of\ second\ node\ i.e.\ ff18\ .The\ \&\ operator\ then\ takes\ the\ address\ where$
$this\ next\ pointer\ is\ stored\ i.e.\ the\ address: ff14, where\ ff18\ is\ stored\ .Hence,$
$current = \&\big((*current) \rightarrow next\big) = ff14\ under\ which\ ff18\ is\ stored.$

```
    while (*current != nullptr)
    {
        s.push(*current);
        current = &((*current)->next);
    }
```

$*current\ is\ not\ equal\ to\ (!=)\ nullptr\ which\ is\ true\ then:$

```
        s.push(*current);
```



$s.push(*current)$, here $*current$ is Node $*$ $\underline{*current}$ pointer

, and dereferencing pointer $(*)$dereference it , to get the

Node $*$ it points to. Here $\underline{Node *}$ points to address: $ff18$, where

$\underline{current}$ points to address : $ff14$. Hence as $\underline{*current}$ pointer gets deferenced by $*$ (dereferencing operator), i.e. now

no longer pointing to address : $ff14$, hence , now, Node $*$ pointed

address is referenced i.e. $ff18$.

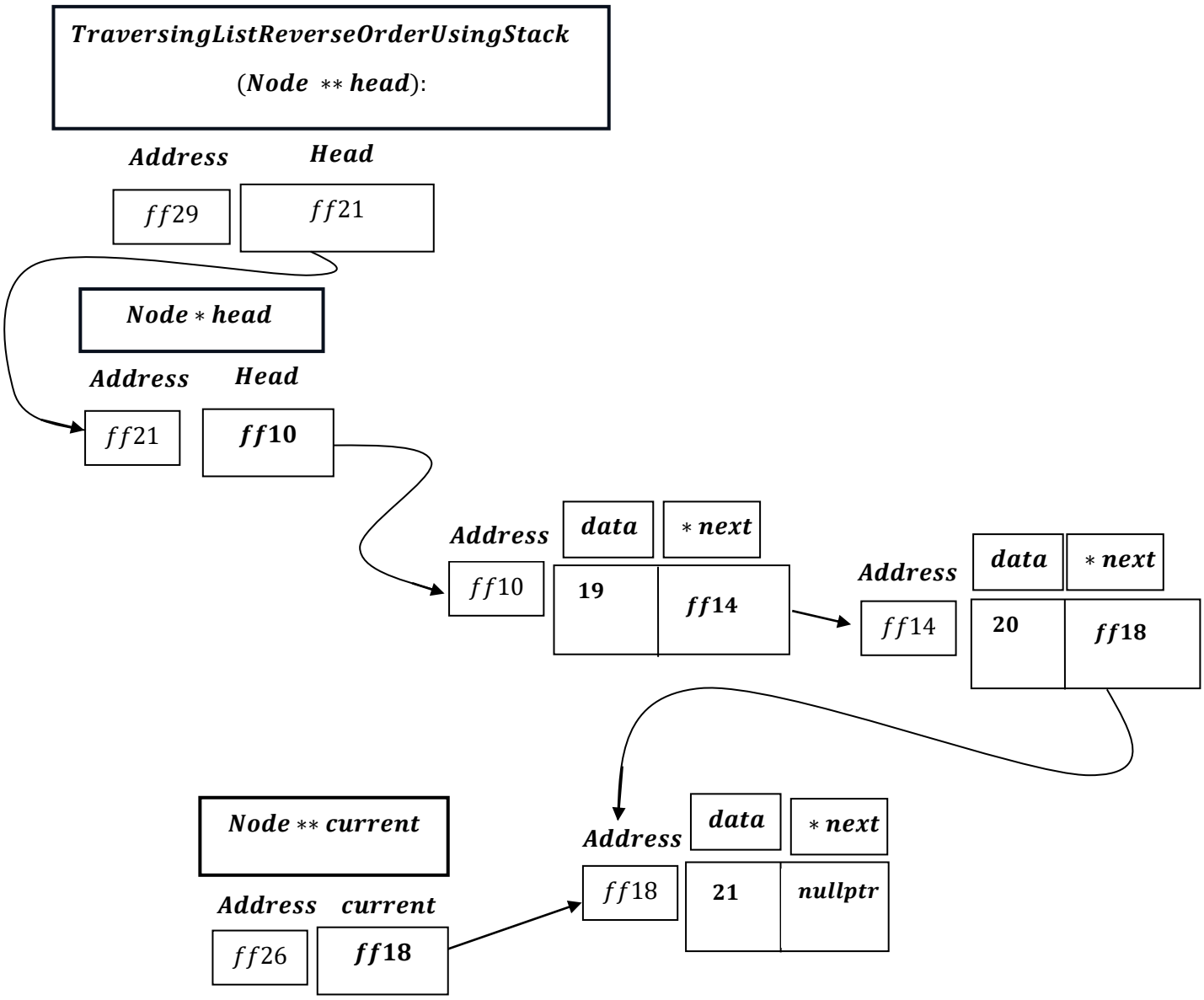
$*$ currrent is $ff18$, which is address of the third node: [Address: $ff18$]

[data: 21| $*$ next: nullptr].The stack will store this address ($ff18$), which

references the third node.

Now updation,

```
        current = &((*current)->next);
```



During the third iteration, (∗ current) dereferences current to get the Node ∗ it points to, which is the third node in the list. The expression (∗ current) → next, where current points to address : ff18, but (∗ current) dereferences first and now Node ∗ points to third node, hence (∗ current) → next, will point to next pointer of third node i.e. ``nullptr`` . The & operator then takes the address where this nullptr is stored i.e. the address: ff18, where ``nullptr`` is stored . Hence, current = &((∗ current) → next) = ff18 under which ``nullptr`` is stored.

```
while (*current != nullptr) {…} //condition checking
```

∗ current, due to conditional checking, hence no braces are needed to prioritize the pointer current variable, i.e. Node ∗ ∗ current pointer, and dereference it , to get the Node ∗ it points to. Here Node ∗ points to ``nullptr``, where current points to address : ff18. Hence as ∗ current pointergets deferenced by ∗ (dereferencing operator), i.e. now no longer pointing to address : ff18, hence , now Node ∗ pointed address is referenced i.e. ``nullptr``.

Now, as soon as we get ``nullptr``, the condition of while loop fails and exits from the loop.

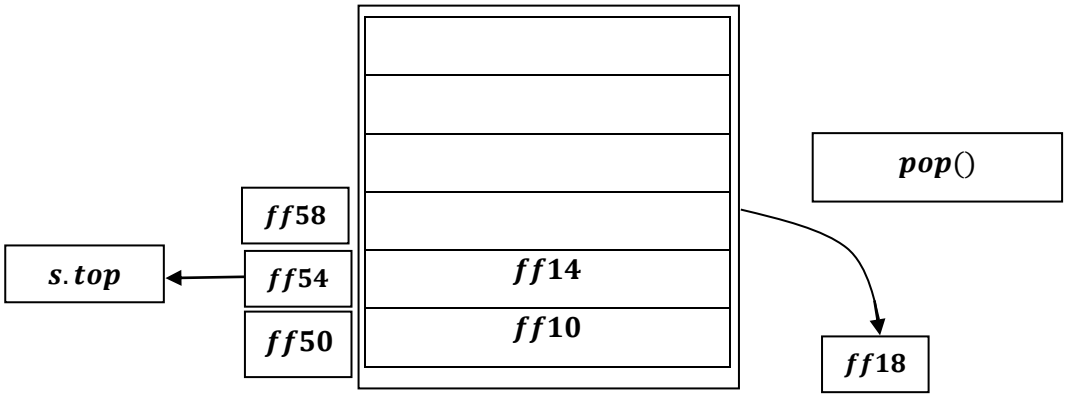
Now, we will print, the data stored in the address that was pushed inside the stack and pop these addresses one by one from the stack.

```
    while (!s.empty())
    {
        cout << s.top()->data << " ";
        s.pop();
    }
```

$s.empty()$ *is not true, then*:

$\rightarrow$ $cout \ll s.top() \rightarrow data \ll$ `` `` ; *accesses* $ff18 \rightarrow data$, *which is* 21.

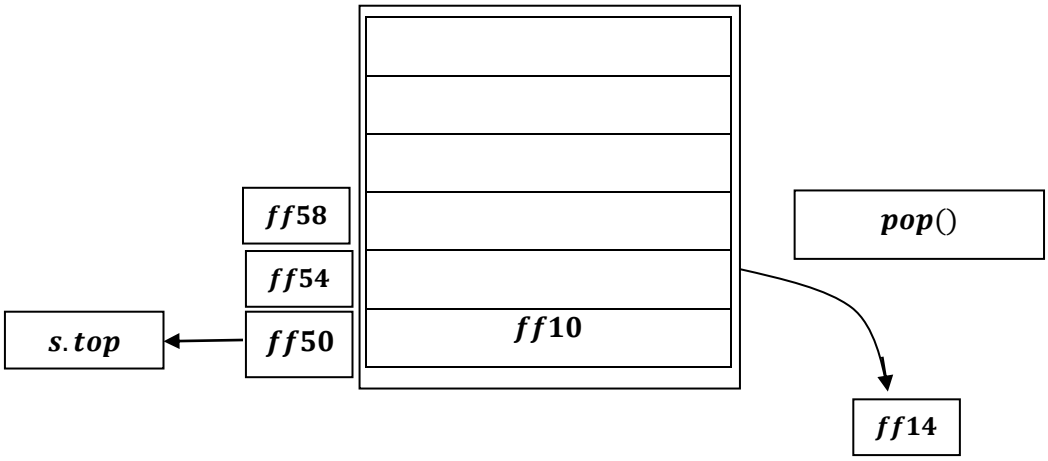
$\rightarrow$ $s.pop()$;



$s.empty()$ *is not true, then*:

$\rightarrow$ $cout \ll s.top() \rightarrow data \ll$ `` `` ; *accesses* $ff14 \rightarrow data$, *which is* 20.

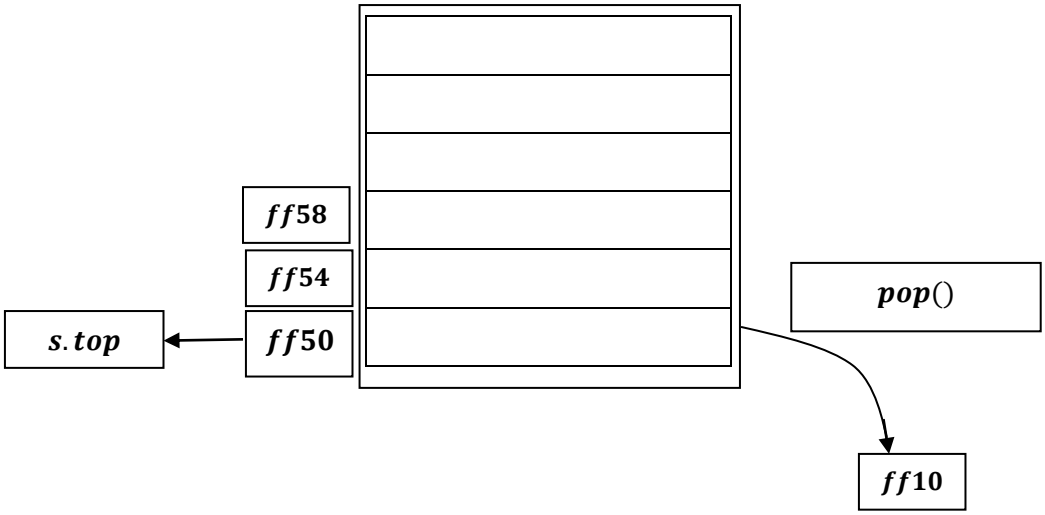
$\rightarrow$ $s.pop()$;



$s.empty()$ *is not true, then*:

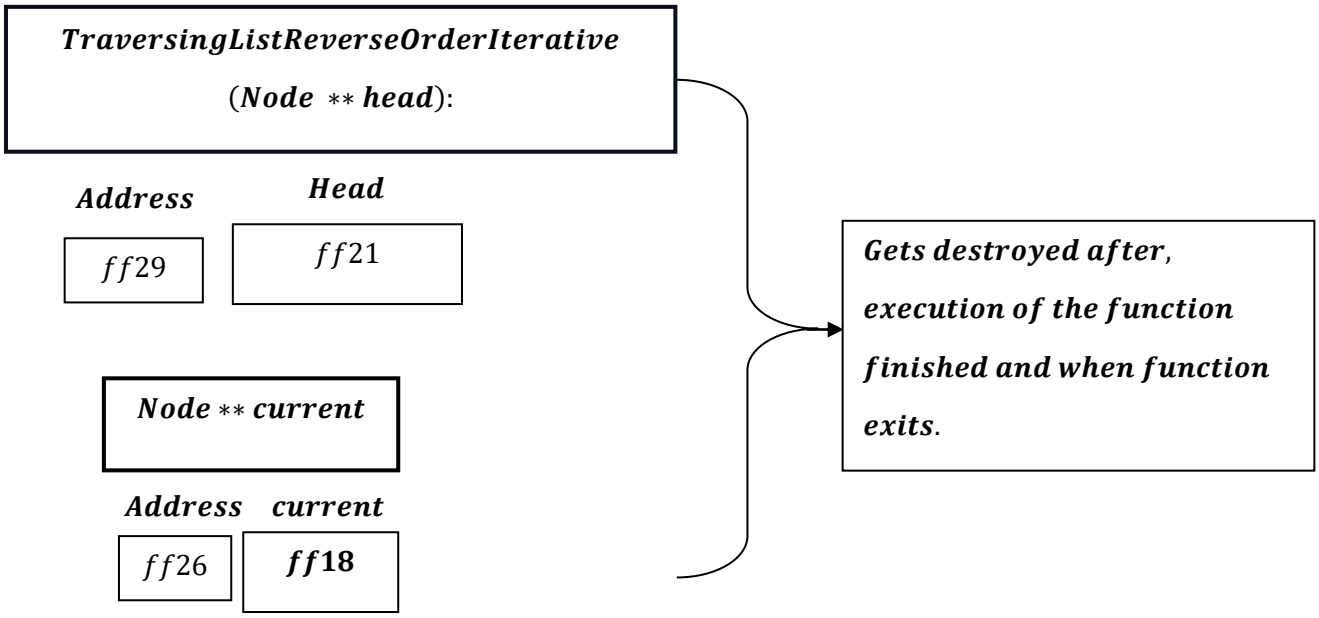
$\rightarrow$ $cout \ll s.top() \rightarrow data \ll$ `` `` ; *accesses* $ff10 \rightarrow data$, *which is* 19.

$\rightarrow$ $s.pop()$;

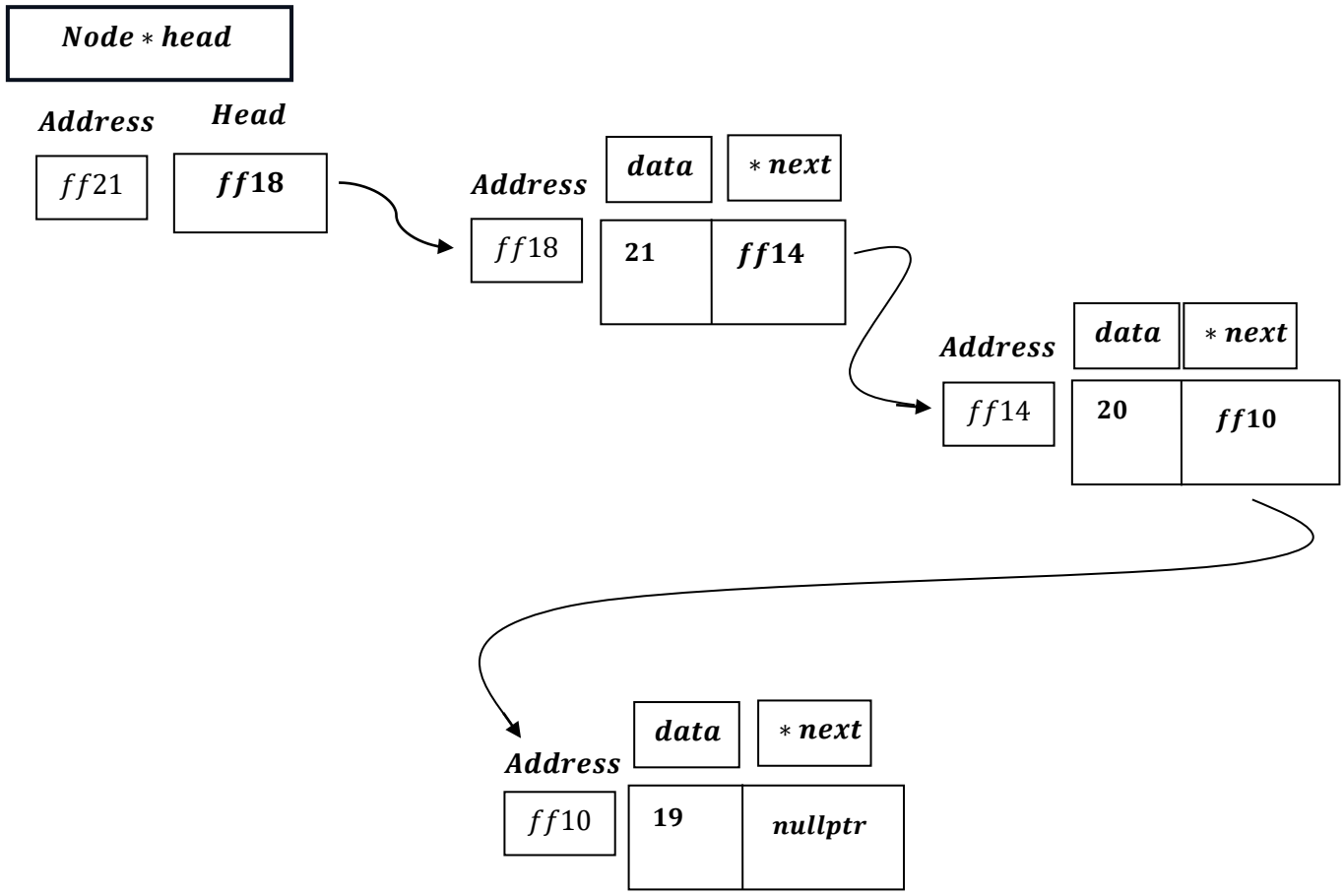


$s.empty()$ *is true, then exit.*

Now, as the function get finished, all temporary variable created earlier in process for the program will get destroyed.



And we are left with:



The function TraverseListReverseOrderUsingStack step by step analysis:

1. ***Function Definition***:

- ***The function TraverseListReverseOrderUsingStack takes a double pointer Node ** head as an argument. This is a pointer to a pointer to the head of a linked list.***

2. ***Check for Empty List***:

- ***The function first checks if the list is empty by checking if * head is nullptr. If it is, it prints*** ``List is empty.``***and returns immediately.***

3. ***Stack Declaration***:

- ***A stack `s` of type `Node * ` is declared. This stack will be used to store the nodes of the linked list.***

4. ***Traverse the List***:

- ***A pointer `current` is initialized to point to `head`. The loop `while (* current != nullptr)` iterates over the linked list.***

- ***Inside the loop, s.push(* current)pushes the address of the node that current points to onto the stack. Here, current is a pointer to a pointer to a node (Node **), so dereferencing current once (* current)gives you the address of the node (Node *). This address is then pushed onto the stack.***

- ***The pointer `current` is then updated to point to the next node in the list using `current = &((* current) → next)`.***

5. ***Print in Reverse Order***:

- ***After all node's address have been pushed onto the stack, another loop `while (! s.empty())` is used to print the nodes in reverse order.***

- ***The top node of the stack is accessed using `s.top()`, and its data is printed by accessing the data through the node's addresses which was pushed onto the stack.***

- ***The node is then removed from the stack using `s.pop()`.***

6. ***End of Function***:

- ***After all nodes have been printed, a newline is printed for formatting.***

$TraverseListReverseUsingStack$

$-Time\ Complexity$.

```
void TraverseListReverseOrderUsingStack(Node **head)
{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    stack<Node *> s;
    Node **current = head;
    while (*current != nullptr)
    {
        s.push(*current);
        current = &((*current)->next);
    }
    while (!s.empty())
    {
        cout << s.top()->data << " ";
        s.pop();
    }
    cout << endl;
}
```

$O(1)$

$O(1)$

$O(1)$

$1 + 1 + 1 + \cdots n\ times$

$1 + 1 + 1 + \cdots n\ times$

$1 + 1 + 1 + \cdots n\ times$

$1 + 1 + 1 + \cdots n\ times$

$O(1)$

<u>$Best\ Case\ (When\ List\ is\ Empty)$</u>

$When\ List\ is\ Empty, then, as\ soon\ as\ it\ enters\ the\ function$: $TraversingListReverseOrderUsingStack, if\ checks\ whether\ head\ pointer\ pointed\ to$ $``nullptr``\ or\ not\ , if\ so\ then\ outputs$: $``List\ Is\ Empty``\ and\ returns\ nothing\ , and\ then$ $exits\ from\ function. This\ takes, constant\ `c`\ amount\ of\ time\ i.e. \Omega(1)$.

$Hence\ when\ List\ is\ Empty, we\ get\ our\ best\ case:\ \Omega(1)$.

$\underline{Worst\ Case\ (When\ List\ Is\ Not\ Empty)}$

$When\ list\ is\ not\ empty\ ,it\ traverse\ whole\ list\ ,assuming\ `n`\ is\ the\ length\ of\ the\ List.$

$\rightarrow Head\ doesnot\ point\ to\ nullptr\ ,when\ List\ is\ not\ empty, hence\ checking\ takes\ O(1)\ time\ complexity.$

$\rightarrow stack < Node *> s; declaration\ takes\ constant\ time\ i.e.\ O(1)\ time\ complexity.$

$\rightarrow Node\ **\ current = head; assignment\ takes\ constant\ time\ i.e. O(1) time\ complexity.$

$\rightarrow Then\ the\ loop\ traverse\ `n`\ no. of\ nodes\ present\ in\ the\ list.$

- $We\ push\ the\ the\ address\ of\ the\ node\ that\ current\ points\ to\ onto\ the\ stack.\ Each\ push\ takes\ constant\ time\ `c`\ and\ continues\ till\ `n`\ times\ i.e.\ 1 + 1 + 1 + \cdots n\ times = 1 \times n = n\ .$

- $Assignment: current = \&((*\ current) \rightarrow next); this\ assignment\ takes\ `n`\ times.$

$Hence\ total\ number\ of\ statements\ in\ the\ loop\ runs\ `n`\ times = O(n + n) = O(2n)$
$= 2 \times O(n) = O(n)\ .$

$\rightarrow Next\ we\ will\ traverse\ through\ a\ loop\ which\ will\ run\ till\ the\ stack\ `s`\ becomes\ empty:$

- $printing\ s.top() \rightarrow data\ i.e. data\ stored\ in\ the\ addresses\ of\ the\ nodes\ get\ printed, each\ takes\ constant\ time\ `c`\ i.e. we\ consider\ 1\ unit\ of\ time\ which\ is: 1 + 1 + 1 + \cdots n = 1 \times n = n\ units\ of\ time\ complexity\ in\ total.$

- $Next, we\ pop\ out\ every\ addresses\ from\ stack\ after\ printing\ data\ from\ each\ addresses, each\ takes\ constant\ time\ `c`\ i.e. we\ consider\ 1\ unit\ of\ time\ which\ is: 1 + 1 + 1 + \cdots n = 1 \times n = n\ units\ of\ time\ complexity\ in\ total.$

$Hence\ total\ number\ of\ statements\ in\ the\ loop\ runs\ `n`\ times = O(n+n) = O(2n)$

$= 2 \times O(n) = O(n)$.

$\rightarrow After\ all\ nodes\ have\ been\ printed, a\ newline\ is\ printed\ for\ formatting. Which\ takes\ 1\ unit\ of\ time\ i.e. O(1)$.

$Hence\ , total\ time\ complexity: -O(1) + O(1) + O(1) + O(n) + O(n) + O(1)$

$= O(n)$.

<u>$Average\ Case(Average\ Case = Worst\ Case)$</u>

$The\ average\ case\ for\ traversing\ a\ linked\ list\ typically\ involves\ traversing\ a\ list\ of\ average\ length. Since\ the\ function\ must\ visit\ each\ node\ to\ print\ its\ data, the\ time\ complexity\ is\ (O(n)), where\ (n)\ is\ the\ number\ of\ nodes\ in\ the\ list.$

$Note\ , by\ relation: \Omega \leq \Theta \leq O\ , we\ get:$

$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Longrightarrow c_1 \times 1 \leq n \leq c_2 \times n$

$Singly\ Linked\ List$	$Cases$	$Time\ Complexity$
$TraversingListReverseOrderUsingStack$	$1. Best\ Case$	$\Omega(1)$
	$2. Worst\ Case$	$O(n)$
	$3. Average\ Case = Worst\ Case$	$\Theta(n)$

****